# Chui Twin: Project Structure and Design Review

A guide to how the system is built, for someone assessing it. State as of October 18, 2026. Companion documents: `docs/technical_appendix.md` (method and results in depth) and `docs/model.md` (the math).

## 1. What the system does

Chui Twin turns a public company into a runnable operating model ("twin") built only from public data. Public data series (TSA checkpoint counts, jet fuel prices, BLS wages, store counts, news tone, Wikipedia attention) are treated as **noisy sensors** of a small set of **hidden operating states**: demand, capacity, price, input cost, labor, sentiment and expansion. One shared engine:

1. estimates those states each quarter from the signals,
2. links states to the KPIs a company reports (revenue growth, margins, comparable sales) through a causal graph,
3. simulates 10,000 paths forward with Monte Carlo, including scenario shocks,
4. attributes every forecast to drivers and to the data series behind them,
5. scores every twin walk-forward against naive baselines, management guidance and analyst consensus.

It was built for the Longaeva AI Hackathon (Sept 30 – Oct 21, 2026). Chipotle (CMG) and Delta (DAL) are the deep, hand-built examples; 15 more twins were built automatically from their tickers.

**Size:** about 6,900 lines of Python across 60 files, an 11-file React dashboard, 70 automated tests, 17 company specs, 29 commits.

**What to look for as an assessor:** whether the engine is genuinely company-agnostic (section 3), whether the backtest is free of lookahead (section 6.2), whether the results are reported honestly (section 10), and whether the code matches the claims (section 9, tests).

---

## 2. Architecture at a glance

```
          PUBLIC SOURCES                          ADAPTERS (one per source)
SEC EDGAR / XBRL / 10-K, FRED, BLS,   --->   fetch -> save raw bytes -> normalize
TSA, GDELT, Wikimedia, sitemaps              to Observation / GroundTruth rows
                                                        |
                                                        v
                                    STORAGE: SQLite (Postgres-portable SQL)
                                    observations, ground_truth, external_forecasts,
                                    raw_snapshots  (every row: observed_at + available_at)
                                                        |
   TEMPLATE (industry)  +  SPEC (company)               |
   templates/operating.yaml  specs/<ticker>.yaml        |
              \            /                            |
               v          v                            v
              ENGINE  (company-agnostic; never names a ticker, state or series)
  estimation/weighted.py  -> hidden states per period (precision-weighted signals)
  estimation/calibration  -> Bayesian regression: states -> KPIs, priors from causal edges
```

```
    simulation/montecarlo   -> 10,000 vectorized paths, shocks, identities, bounds
    attribution/            -> driver waterfall, leave-one-signal-out counterfactuals
    evaluation/             -> walk-forward backtest, baselines, CRPS, interval coverage
    sealing.py              -> canonical JSON + SHA-256 (+ OpenTimestamps proof)
                                          |
              +-----------------------------------+-----------------+
              v                                   v                 v
     API (FastAPI, read-only)              SCRIPTS (CLI)          ONBOARDING
     forecasts, scenarios, scorecard,      daily job, backtest,   ticker -> SEC -> pack -> spec
     chat assistant (Claude + tools)       forecast/seal, resolve  (+ LLM reads the 10-K)
              |
              v
     WEB dashboard (React + Recharts)
```

**The key separation:** industry structure lives in YAML templates, company detail in YAML specs, data access in adapters. The engine only knows that a template has named states, edges between them and KPIs built from them. Adding a company is a spec file; adding an industry is a template or pack file. Neither requires engine code.

---

## 3. Design rules and how the code enforces them

| Rule | What it means | Enforced by |
|---|---|---|
| 1. No company code in the engine | `engine/` never contains tickers, state names, KPI names or series prefixes | `tests/test_engine_no_company_code.py` greps `engine/`; `tests/test_template_agnostic.py` runs estimation and simulation end to end on `templates/_dummy.yaml`, a template with made-up names |
| 2. Every number is traceable | Every observation, ground-truth value and forecast carries a `source_ref` (filing accession plus exhibit, URL plus quote, or raw file path) | `Observation`, `GroundTruth`, `ExternalForecast` dataclasses require it; DB columns are `NOT NULL` |
| 3. Point-in-time correctness | Every row stores `observed_at` (the period it describes) and `available_at` (when it could first be known); backtests use only rows with `available_at <= as_of` | `tests/test_point_in_time.py`; the fit takes an `as_of` and filters every input |
| 4. Raw first, parse later | Every fetch saves the untouched response to `data/raw/<source>/<date>/` with a `.meta.json` sidecar (URL, params, time, SHA-256) before parsing | `adapters/http.py::save_raw` is the only path from network to parser |
| 5. Legal data only | Public sources, no logins, robots.txt and terms checked, polite rate limits | `provenance.md` logs each source; `PoliteClient` enforces spacing and backoff; never bypasses blocks |
| 6. Plan before large refactors | Changes touching more than 3 files are proposed first | Process rule (`CLAUDE.md`) |

---

## 4. Repository map

```
chui-twin/
  README.md               one-command demo, results summary
  CLAUDE.md               project rules and implementation notes
  provenance.md           every data source: access, terms, what it informs, available_at rule
  pyproject.toml          Python 3.11+, numpy, pydantic, pyyaml, requests (+ api, dev, seal extras)
```

```
conftest.py              test setup

engine/                  COMPANY-AGNOSTIC CORE (~2,000 lines)
  observation.py         RawSnapshot, Observation, GroundTruth, ExternalForecast records
  primitives.py          Template, PrimitiveDef, EdgeDef, KPIDef (pydantic); load_template()
  spec.py                Spec (pydantic), resolve(spec, template) -> ResolvedSpec, identities
  periods.py             period labels ("2024Q3"), shift, span, period_of(date)
  formula.py             safe arithmetic over KPI names ("transactions + check")
  twin.py                Twin.fit(as_of) -> FittedTwin.forecast(n_paths, shocks) -> Forecast
  scenarios.py           shocks defined on data series instead of hidden states
  sealing.py             canonical JSON + SHA-256; verify()
  estimation/
    weighted.py          signals -> features -> state means/variances (v0, precision-weighted)
    calibration.py       Bayesian linear regression states -> KPIs, priors from edges; lags; anchors
  simulation/
    montecarlo.py        vectorized NumPy paths; AR(1) states; shocks with half-lives; identities; bounds
  attribution/
    waterfall.py         decompose each KPI's expected value into driver and signal terms
    counterfactual.py    leave-one-signal-out: Y(all) - Y(without signal i)
  evaluation/
    backtest.py          walk-forward refit at each release; scorecard tables; skill
    baselines.py         naive_last, seasonal_naive, guidance, consensus
    metrics.py           CRPS (Gaussian and empirical), MAE, interval coverage

adapters/                DATA ACCESS, reusable across companies
  base.py                Adapter ABC: fetch(params, start, end) -> [RawSnapshot]; normalize() -> [Observation]
  __init__.py            registry: series prefix -> adapter ("fred:", "bls:", "tsa:", "wiki:", "gdelt:", "sitemap:")
  http.py                PoliteClient (spacing, retries, backoff, User-Agent); save_raw / read_raw
  edgar.py               SEC submissions, filing documents, XBRL company facts; release text extraction;
                         guidance extraction; quarterly facts with point-in-time ordering
  fred.py  bls.py  tsa.py  wikimedia.py  gdelt.py
  scrapers/sitemap.py    generic sitemap URL counter with Wayback Machine history (config: sitemaps.yaml)
  series_catalog.yaml    human-readable series names (display only; the engine never reads it)

templates/
  operating.yaml         the industry template: 7 states, default edges with priors, KPI structure
  _dummy.yaml            test-only template with invented names

specs/
  cmg.yaml  dal.yaml     hand-built twins
  auto/*.yaml            factory-built twins (14) + Carnival (LLM); <ticker>.onboarding.json = LLM audit trail
  experiments/llm/       LLM specs for the accuracy comparison (kept out of the portfolio)
  external/              hand-entered consensus with source URLs and quotes

onboarding/
  industry_packs.yaml    5 packs (restaurants, airlines, hotels, homebuilders, general retail), chosen by SIC code
  factory.py             ticker -> SEC lookup -> pack -> verified Wikipedia + news query -> XBRL rules -> spec
  llm.py                 LLM reads the 10-K, proposes drivers/KPIs/edges with quotes; code verifies

prompts/                 versioned LLM prompts: chat_v1.md, onboard_spec_v1.md
shocks/library.yaml      shock profiles measured from real events (generated)
forecasts/sealed/        sealed forecasts (.json) and OpenTimestamps proofs (.ots)
migrations/001_init.sql  schema, portable SQL (SQLite in dev, Postgres-ready)
storage/db.py            SQLite store: save/load observations, ground truth, external forecasts

api/
  main.py                read-only FastAPI: /twins, /twins/{t}/forecast, /history, /scorecard,
                         /portfolio/shock, /shocks, /series/labels
  chat.py                /chat: Claude with tools over the same functions (every number from a tool)
  serve.py               one process: API under /api, built dashboard at /

web/src/                 React 19 + Recharts + Tailwind dashboard
  App.tsx  Portfolio.tsx  TwinView.tsx  EconomyShock.tsx  ChatPanel.tsx  Charts.tsx  api.ts  ...

scripts/
  run_daily.py / .cmd    scheduled job: refresh every series, ground truth, timestamp proofs
  fill_missing.py        slow refetch of missing or stale series (rate-limited sources)
  ingest.py              ingest signals and ground truth for one spec
  make_twin.py           ticker(s) in, scored twin(s) out, portfolio scorecard (--llm for LLM onboarding)
  backtest.py            walk-forward backtest for one spec, incl. guidance and consensus comparisons
  forecast.py            current forecast, waterfall, scenarios; --seal writes and timestamps it
  resolve_sealed.py      verifies hashes, accepts outcomes only if public after the seal, scores
  validate_ground_truth.py  automatic checks on every reported value + manual sample
```

```
    shock_library.py       measures shock profiles from filings
    ots.py                 OpenTimestamps stamp / upgrade / info
    experiments/           e.g. jolts_labor.py (tested, rejected)
    reports/               PDF renderers (this document, the appendix, status reports)

  tests/                   70 tests; fixtures are saved real responses; no network
  docs/                    model, appendix, scorecard, experiments, validation, sealed forecasts, coverage, ...
  data/                    chui.sqlite + raw/ (gitignored; ~550 MB of raw responses)
```

---

# 5. Layer by layer

## 5.1 Adapters (`adapters/`)

Adapters fetch data and return normalized rows. They know nothing about states or companies; the spec maps series to states. Series names encode their query, and the registry routes by prefix: `fred:DJFUELUSGULF`, `bls:CES4348100003`, `tsa:checkpoint_throughput`, `wiki:Delta_Air_Lines`, `gdelt:tone:delta air lines`, `sitemap:chipotle_stores`.

```
class Adapter(ABC):
    source: str
    prefix: str
    def fetch(self, params: dict, start: date, end: date) -> list[RawSnapshot]: ...
    def normalize(self, snapshots: list[RawSnapshot]) -> list[Observation]: ...
```

| Adapter | Source and access | Frequency | `available_at` rule |
|---|---|---|---|
| `edgar.py` | SEC submissions, 8-K exhibits, 10-K, XBRL company facts; declared User-Agent from env, ≤5 req/s | per filing | EDGAR acceptance time; XBRL: filing date + 1 day |
| `fred.py` | fredgraph CSV, no key | daily to quarterly | period end + 2d / 7d / 45d / 100d by frequency |
| `bls.py` | BLS API v2, no key (25 queries/day) | monthly | month end + 40d |
| `tsa.py` | TSA passenger volumes HTML table | daily | day + 2d |
| `wikimedia.py` | Pageviews REST API | daily | day + 2d |
| `gdelt.py` | GDELT DOC API timeline tone; 1 request per 10 s, fails fast on 429 | daily | day + 1d |
| `scrapers/sitemap.py` | public sitemaps (counts of URLs only) + Wayback Machine history | daily | fetch / capture time |

`http.py::PoliteClient` enforces minimum spacing between requests, retries with exponential backoff, honors `Retry-After`, and refuses to retry on 401/403 (never bypasses blocks). `save_raw` writes the bytes and a sidecar before any parser runs, so a broken parser can be fixed and rerun on saved data.

`edgar.py` is the largest adapter: it extracts Chipotle KPIs from earnings-release text with spec-defined rules (each value keeps its source sentence), extracts management guidance (including worded guidance such as "mid-teens" mapped to a range), derives implied quarterly guidance from annual guidance, and builds quarterly XBRL facts with point-in-time ordering (the first filing that reported a value wins; Q4 derived as full year minus nine months when needed).

## 5.2 Storage and schema (`migrations/001_init.sql`, `storage/db.py`)

| Table | Key | Purpose |
|---|---|---|
| `raw_snapshots` | sha256 | every fetched file: source, time, path, URL, params |
| `observations` | (entity, series, observed_at, available_at) | signal values; revisions kept as separate rows |
| `ground_truth` | (entity, period, kpi, available_at) | reported KPIs with `source_ref`, confidence, verified flag |
| `external_forecasts` | (entity, period, kpi, source, available_at) | guidance and consensus |
| `runs`, `states`, `forecasts` | run id | forecast runs, estimated states, forecast quantiles, seal hash |

`available_at` is part of the primary key, so a revised value is a new row rather than an overwrite; a backtest can see exactly what was known on any date. Current contents: 164,221 observations of 68 series; 2,564 reported KPI values for 17 companies; 56 guidance and 6 consensus values; 362 snapshot records.

### 5.3 Templates and specs (`templates/`, `specs/`, `engine/spec.py`)

A **template** defines the states (each with a persistence), default causal edges with priors, and default KPI structure. The `operating` template covers businesses whose revenue is volume times price served by physical capacity (restaurants, retail, airlines, hotels, homebuilders, transport).

A **spec** picks a template and declares the company's KPIs, signals (series → state, with sign, lag, confidence, half-life, transform) and edge overrides. Excerpt from `specs/dal.yaml`:

```
ticker: DAL
template: operating
exclude_from_calibration: ["2020Q1:2022Q4"]      # COVID regime break; still forecast and scored
kpis:
  revenue_growth: {unit: pct, target: change, anchor: guidance}   # models the surprise vs guidance
  op_margin: {unit: pct, lag: {prior_mean: 0.3, prior_sd: 0.2}, seasonal_lag: {prior_mean: 0.6, prior_sd: 0.2}}
signals:
  - {series: "tsa:checkpoint_throughput", primitive: demand, confidence: 0.6, half_life_days: 30}
  - {series: "fred:DJFUELUSGULF", primitive: input_cost, confidence: 0.7, half_life_days: 30}
edges:
  - {from: demand, to: revenue_growth, prior_mean: 0.8, prior_sd: 0.3}
```

`spec.py` validates the spec with pydantic (unknown fields rejected), merges it with the template, checks that every state and KPI exists, parses accounting identities (must be linear, no constant), orders states topologically, and produces a `ResolvedSpec` with a `spec_hash()` used to identify runs and seals. Each KPI can be a **level** (optional last-quarter and same-quarter-last-year terms), a **change** since the last report, or an **anchored surprise** relative to guidance or consensus. Why each structure was chosen is recorded as a comment in the spec.

### 5.4 Engine (`engine/`)

`twin.py` is the pipeline: `Twin(spec, observations, ground_truth, external).fit(as_of)` returns a `FittedTwin`, whose `forecast(n_paths, shocks)` returns a `Forecast` (quantiles per KPI and period, expectation gaps versus guidance/consensus, the driver waterfall, and a serializable record for sealing).

| Module | What it does |
|---|---|
| `estimation/weighted.py` | Each series becomes a decay-weighted level, then a year-over-year change (or level), standardized over history available at `as_of`. Each state has a N(0,1) prior; each signal is a reading with precision c/(1−c) · 0.5^(staleness/half-life). The state mean is the precision-weighted average, so each signal's share is exact and stored. |
| `estimation/calibration.py` | Bayesian linear regression from states to each KPI with priors from the edges (standardized units), optional lag and seasonal-lag terms, change targets and anchors. Excluded periods are left out of calibration only. |
| `simulation/montecarlo.py` | Fully vectorized NumPy: states evolve as AR(1) with edge-propagated innovations; shocks decay with a half-life; coefficient uncertainty is sampled; identities enforced by minimum-variance projection; physical bounds clip paths and the clipped share is reported. 10,000 paths take about 0.03 s. |
| `attribution/waterfall.py` | Splits each KPI's expected value into baseline, lag/anchor terms and driver terms, each driver into its signals. |
| `attribution/counterfactual.py` | Leave-one-signal-out: how much the forecast moves without each signal. |
| `evaluation/backtest.py` | Walk-forward: for each test period, fit as of just before that period's release, forecast, score against the reported value and baselines. |
| `evaluation/baselines.py` | `naive_last` (same as last quarter), `seasonal_naive` (same quarter last year), `guidance`, `consensus` (latest published before the release). |
| `evaluation/metrics.py` | CRPS, MAE, 80% interval coverage. Skill = 1 − CRPS(twin)/CRPS(baseline). |
| `sealing.py` | Canonical JSON (sorted keys, compact separators; LF line endings pinned by `.gitattributes` so hashes stay byte-exact) and its SHA-256. |

### 5.5 Onboarding (`onboarding/`)

**Rules-only factory (`factory.py`):** ticker → CIK, name and SIC code from SEC → industry pack by SIC → shared industry signals + two company signals: the company's Wikipedia article (accepted only if its infobox lists the ticker; an unverified name match once mapped D.R. Horton to "Horton (Dr. Seuss)") and a GDELT news query built from the verified title → XBRL ground-truth rules (revenue growth, operating margin) that work for any SEC filer → `specs/auto/<ticker>.yaml`.

**LLM onboarding (`llm.py`, `--llm`):** Claude Sonnet 5.5 (model from config) reads the latest 10-K and a menu of what the system can use (states, packs, fetchable series, the XBRL concepts this company actually reports). It returns structured JSON (validated by pydantic) proposing the pack, up to 4 extra drivers, up to 2 extra XBRL KPIs, edge strengths and operating metrics, **each with a quote from the 10-K**. Code then drops anything whose quote is not in the filing, whose series cannot be fetched, whose XBRL concept lacks 16 quarters or is no longer reported, or whose state or KPI is unknown. Prompt and response are saved raw; `<ticker>.onboarding.json` records every accepted item with its quote and every rejection with its reason.

### 5.6 API, chat and dashboard

`api/main.py` is read-only and returns sources with every number. Scenarios come in as query parameters (`?shock=state:period:size[:half_life]`); `/portfolio/shock` moves one shared series and returns every exposed twin's response. `api/chat.py` gives Claude strict-schema tools that wrap the same functions (list twins, forecast with shocks, economy shock, scorecard, read docs), so it cannot quote a number the system did not produce; the system prompt is versioned in `prompts/chat_v1.md`; refusals fall back server-side. The React dashboard

shows the portfolio scorecard, economy-shock buttons, per-twin forecast bands with guidance, driver bars linked to sources, state history, scenario presets with measured half-lives, and the chat panel.

### 5.7 Operations

`scripts/run_daily.cmd` runs at 07:00 via Windows Task Scheduler: refreshes every series used by any spec (each fetched once even when shared), refreshes ground truth, then `fill_missing.py --stale 3` slowly refetches anything rate-limited, and timestamp proofs are upgraded once Bitcoin confirms them. Output goes to `data/daily.log`.

---

## 6. One forecast, end to end (Chipotle)

1. **Data.** The daily job has saved FRED restaurant sales and CPI, BLS restaurant wages, beef and chicken prices, Chipotle's Wikipedia views, GDELT news tone, and the store and job counts from Chipotle's sitemaps. EDGAR releases provide 59 quarters of comps, transactions, check and restaurant margin, each with its source sentence.
2. **Point in time (6.2).** `fit(as_of=now)` drops every row whose `available_at` is later than now. A September CPI print counts as known only 45 days after month end.
3. **States.** Signals are fused into seven states per quarter; for Q3 2026, sentiment is −1.16 sd (news tone fell in July–August).
4. **Calibration.** Comps are modeled as an anchored surprise relative to guidance (consensus when available, published before the release); transactions and check are tied by the identity comps = transactions + check.
5. **Simulation.** 10,000 paths; each path satisfies the identity and bounds.
6. **Output.** Comps median +2.94%, 80% range 0.64 to 5.15; the waterfall shows the guidance anchor +2.65, the sentiment drag −0.64, small demand and price terms, each linked to its series and raw files.
7. **Seal.** `forecast.py --seal` writes canonical JSON, records SHA-256 `fabe5e81…`, and submits it to OpenTimestamps for a Bitcoin anchor. It resolves when Chipotle reports on October 28.

---

## 7. Data sources and coverage

Six signal sources plus SEC filings, all public, all logged in `provenance.md` with access method, terms and `available_at` rule. 17 companies across 5 industry packs (restaurants, airlines, hotels, homebuilders, general retail) plus a cruise line mapped to the hotels pack by the LLM. Shared series serve many twins: one TSA series feeds every airline and hotel; jet fuel feeds every airline and both cruise lines tested.

One legal judgment call is recorded openly: Chipotle's site terms say "personal, noncommercial use"; the system stores only counts of public store and job URLs and never republishes pages (accepted risk, 2026-10-01). Chipotle menu prices are not scraped because they sit behind an API key.

---

## 8. Evaluation protocol

- **Walk-forward from 2017Q1:** at each historical earnings release, refit on data available then and forecast.

- **Exclusions:** COVID quarters (2020Q1–2021Q4) excluded from scoring for everyone; travel twins also exclude calibration through 2022Q4 (stated in the spec).
- **Baselines:** same as last quarter, same quarter last year, management guidance, analyst consensus.
- **Metric:** CRPS on the full forecast distribution; skill versus each baseline; 80% interval coverage.
- **Selection bias, stated:** the hand-built twins' KPI structures were chosen on the same backtest. The clean tests are the 14 rules-only factory twins (untuned industry defaults) and the sealed live forecasts.

---

## 9. Tests (70, no network)

| File | Tests | Guards |
|---|---|---|
| `test_engine_no_company_code.py` | 2 | rule 1: no tickers, state, KPI or series names in `engine/` |
| `test_template_agnostic.py` | 6 | engine runs end to end on a template with invented names |
| `test_point_in_time.py` | 4 | rule 3: nothing available after `as_of` influences a fit |
| `test_adapters.py` | 7 | every adapter parses a saved real response (`tests/fixtures/raw/`) |
| `test_spec.py` | 8 | spec validation, merging, identities, errors |
| `test_simulation_and_metrics.py` | 5 | simulation correctness and speed, CRPS |
| `test_day2.py` | 18 | sitemap counter, GDELT, factory helpers, seasonal lag, change targets |
| `test_onboarding_llm.py` | 7 | LLM checks: verbatim quotes, unknown series/states, stale XBRL, schema |
| `test_resolve_sealed.py` | 3 | hash check, outcome-after-seal rule, scoring |
| `test_validate_ground_truth.py` | 1 | text-span checks on real release wording |
| `test_chat.py` | 3 | chat loop with a fake Claude client |
| `test_api.py` | 1 | API smoke test (skipped without a local database) |

Run with `python -m pytest`. LLM and chat tests use hand-written stand-ins for model replies, so they need no key.

---

## 10. Results (rescored October 18, 2026)

| Result | Value |
|---|---|
| KPI series beating "same as last quarter" (CRPS) | 25 of 37 |
| KPI series beating "same quarter last year" | 35 of 37 |
| Factory twins (no tuning) beating "last quarter" | 21 of 30 series |
| Delta revenue growth vs Delta's guidance | skill +0.29 (14 quarters), twin better |
| Chipotle comps vs Chipotle's guidance | skill +0.31 (20 quarters), twin better |
| Chipotle comps vs analyst consensus | skill −0.37 (6 quarters), consensus better |
| Ground-truth values checked automatically | 2,564, 0 failures, 24 outliers flagged (real events) |

Airlines are strongest (direct, timely sensors). Margins beat naive more often than revenue. Restaurant and hotel revenue roughly ties naive. Full table: `docs/portfolio_scorecard.md`.

**Sealed live forecasts.** Delta Q3 (sealed Oct 2, Bitcoin block 969637): operating margin 6.9 vs actual 7.2, a hit; revenue growth 15.2 vs actual 21.1, a miss outside the 80% range because GAAP revenue included refinery sales that the guidance anchor excluded (ex-refinery growth was 15.7%). Chipotle Q3 (sealed Oct 18): comps +2.94%, resolves Oct 28. Context recorded: management guided Q3 comps around +1% on its earnings call, which the twin did not read (it reads guidance from press releases only).

**Experiments kept even when negative** (`docs/experiments.md`): JOLTS hiring data did not improve the backtest and was not adopted; LLM onboarding added coverage and useful new KPIs but no accuracy gain on shared KPIs.

---

## 11. Technology

| Layer | Stack |
|---|---|
| Core | Python 3.11+, NumPy, pydantic v2, PyYAML, requests |
| API | FastAPI, uvicorn |
| LLM | Anthropic SDK; model from env (`CHUI_ONBOARD_MODEL`, `CHUI_CHAT_MODEL`, default `claude-sonnet-5-5`); structured JSON outputs |
| Storage | SQLite in development; schema written in portable SQL for Postgres |
| Web | React 19, Vite, Recharts, Tailwind 4 |
| Timestamping | OpenTimestamps client |
| PDFs | Python-Markdown + headless Chrome/Edge |

## 12. How to run it

```
pip install numpy "pydantic>=2" pyyaml requests fastapi uvicorn anthropic markdown
export CHUI_SEC_USER_AGENT="Your Name you@example.com"      # SEC requires a contact
python scripts/ingest.py specs/cmg.yaml --signals --ground-truth
python scripts/make_twin.py SBUX UAL                        # factory: ticker in, scored twin out
python scripts/make_twin.py CCL --llm                       # + LLM reads the 10-K (needs ANTHROPIC_API_KEY)
python scripts/backtest.py specs/dal.yaml                   # walk-forward with guidance comparison
python scripts/forecast.py specs/cmg.yaml --seal            # current forecast, sealed and timestamped
cd web && npm install && npm run build && cd ..
python -m uvicorn api.serve:app --port 8000                 # dashboard at http://localhost:8000
python -m pytest                                            # 70 tests, no network
```

---

## 13. Known limitations and technical debt

### Modeling

- Short histories (26–31 scored quarters per KPI); skill differences under about 0.05 are noise.
- Causal edges come from priors and are validated, not discovered.
- Interval coverage is uneven (some twins too wide at 1.00; Delta revenue narrow at 0.67).
- Structure chosen with hindsight for hand-built twins and LLM-read specs (stated in the docs).
- Guidance is read from earnings releases only; guidance given on calls is missed.

**Data**

- GDELT rate-limits hard; news tone can lag days, and GDELT's own data has gaps (11 days missing in Chipotle Q3).
- Lennar's operating-margin XBRL stops in 2022; homebuilders tag no operating income, so DHI and PHM use pre-tax margin (edited by hand; regenerating those specs undoes it).
- The database still holds ground-truth rows for KPIs from the LLM experiment and an early Carnival run that are not in any portfolio spec (harmless, but untidy).

**Engineering**

- SQLite only in practice; Docker and Postgres are planned but not set up on the dev machine.
- `pyproject.toml` lists the core dependencies but not `anthropic`, `markdown`, `fastapi` extras used by scripts.
- Some planned adapters in the original plan (Google Trends, USDA AMS, EIA) were not built; FRED covers the commodity prices needed so far.
- The dashboard labels the LLM-onboarded twin as "factory" (the scorecard and CLI label it "factory+llm").
- Single-machine scheduled job (Windows Task Scheduler); no alerting if it fails.

---

## 14. Where to look to check a claim

| Claim | Check |
|---|---|
| Engine has no company code | `tests/test_engine_no_company_code.py`, `tests/test_template_agnostic.py`, `templates/_dummy.yaml` |
| No lookahead | `tests/test_point_in_time.py`; `available_at` rules in `provenance.md`; `engine/twin.py::fit` |
| Every number traceable | any row's `source_ref`; driver bars in the dashboard; `data/raw/` sidecars |
| Results | `docs/portfolio_scorecard.md`; rerun `scripts/make_twin.py --specs specs/*.yaml specs/auto/*.yaml` |
| Sealed forecasts made before outcomes | `forecasts/sealed/*.json` + `.ots` on opentimestamps.org; `scripts/resolve_sealed.py` |
| Extracted values are right | `docs/validation_report.md`, `docs/validation_sample.md` |
| LLM output is checked | `onboarding/llm.py::check`, `specs/auto/ccl.onboarding.json`, `tests/test_onboarding_llm.py` |
| Negative results reported | `docs/experiments.md` |